universidad San Carlos de Guatemala

Centro Universitario de Occidente División de Ciencias de la Ingeniería Ingeniería

Lenguajes formales de programacion sección “A”

Ing. Daniel Gonzales

Segundo semestre 2026

**ESTUDIANTE:**
**WILLIAN ALEXIS IXCOT AJCA**

**CARNER: 202430477**

**PROYECTO 2, 28 DE OCTUBRE 2026**

# MANUAL TECNICO

# Introduccion

PromptZal es un lenguaje orientado a la construcción estructurada de instrucciones o prompts destinados a herramientas de inteligencia artificial. Su propósito es permitir que un programador describa, mediante una sintaxis corta y organizada, el contexto de trabajo, los datos disponibles, las operaciones que desea realizar y los resultados que espera obtener.

El Proyecto 2 completa el proceso de compilación de PromptZal. A diferencia de las etapas anteriores, en las que el trabajo se concentró principalmente en el análisis léxico y en la formalización del lenguaje mediante un Autómata Finito Determinista, esta nueva etapa incorpora el análisis sintáctico, la construcción de un Árbol de Sintaxis Abstracta, la tabla de símbolos, el análisis semántico y la generación automática del promptzal final.

El compilador recibe como entrada un programa escrito en un archivo con extensión .pz. El contenido es procesado inicialmente por un analizador léxico generado con JFlex, el cual produce una secuencia de tokens con información de tipo, lexema, fila y columna. Posteriormente, dichos tokens son enviados a un parser descendente recursivo implementado manualmente en Java.

Mientras el parser reconoce la estructura del programa, construye simultáneamente un Árbol de Sintaxis Abstracta o AST. Esta estructura representa en memoria las directivas, agentes, variables, comandos, expresiones, conectores, ejecuciones y exportaciones que forman el programa.

Una vez construido el AST, el sistema realiza el análisis semántico. Durante esta fase se verifican reglas como la declaración previa de variables, la existencia de agentes utilizados mediante EJECUTAR, la ausencia de agentes duplicados y la validez de las variables solicitadas mediante EXPORTAR. Al mismo tiempo se construye la tabla de símbolos con información sobre nombres, tipos, ámbitos y líneas de declaración.

Finalmente, el generador de PromptZal recorre directamente el AST y construye el prompt final en lenguaje natural. El resultado puede visualizarse dentro de la interfaz gráfica y guardarse como archivo .md o .txt.

La aplicación incorpora además una interfaz gráfica desarrollada con Java Swing y organizada visualmente como un entorno de desarrollo integrado. Desde ella es posible escribir, abrir y guardar programas .pz, compilar el contenido, revisar tokens, errores, estadísticas y símbolos, visualizar el prompt generado y acceder a los reportes del compilador.

# REQUISITOS Y HERRAMIENTAS UTILIZADAS

Las principales herramientas y tecnologías utilizadas son las siguientes:

Lenguaje de programación:
Java

Versión utilizada:
Java JDK 25

Entorno de desarrollo:
Apache NetBeans

Tipo de aplicación:
Aplicación de escritorio

Biblioteca gráfica:
Java Swing

Generador de analizador léxico:
JFlex 1.9.1

Sistema de construcción:
Apache Maven

Herramienta de generación gráfica:
Graphviz

Formato de reportes:
HTML,

# 1.ARQUITECTURA GENERAL DEL SISTEMA

La arquitectura de PromptZal fue diseñada separando las responsabilidades principales del compilador en diferentes paquetes y componentes. Esta separación permite mantener independiente la lógica de análisis del código encargado de la interfaz gráfica, los reportes y el manejo de archivos.

A nivel general, el sistema se divide en dos áreas principales:

• Backend: contiene todas las fases del compilador y las herramientas de procesamiento.

• Frontend: contiene la interfaz gráfica y los componentes encargados de presentar información al usuario.

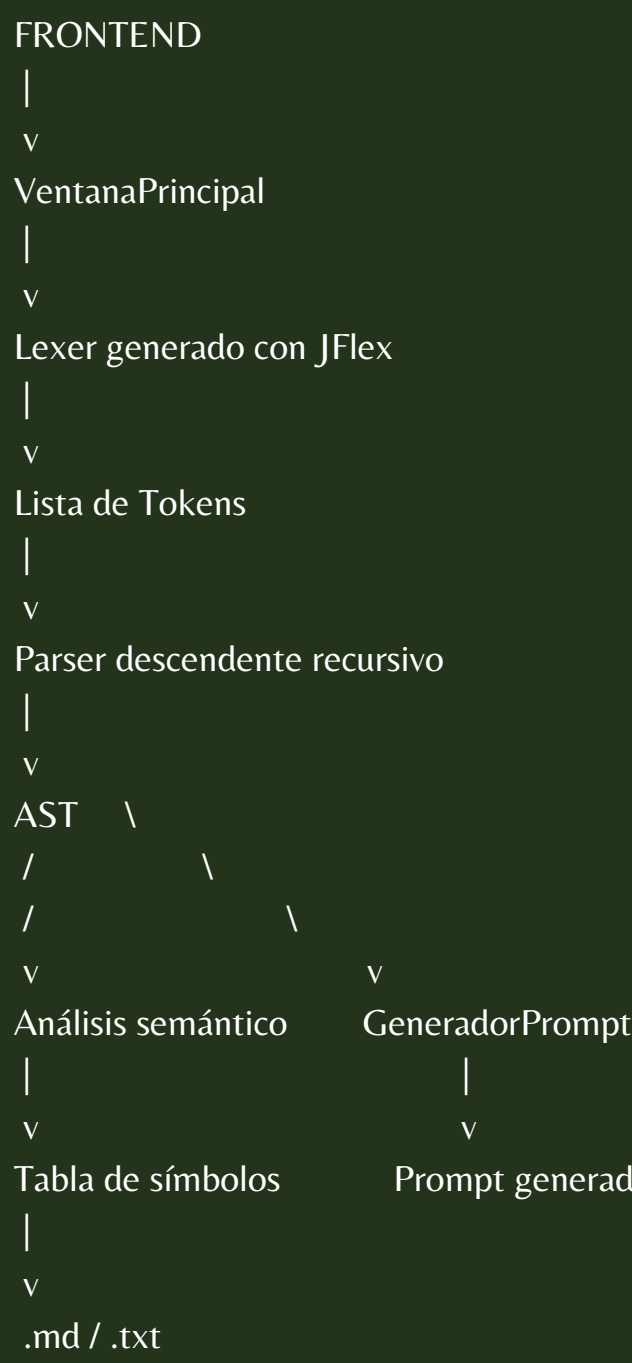
El flujo principal inicia cuando el usuario escribe o abre un archivo .pz desde la interfaz. El contenido se envía al Lexer generado por JFlex. El Lexer transforma el texto en tokens y registra cualquier error léxico encontrado.

La lista de tokens es entregada al Parser. El Parser aplica la gramática libre de contexto mediante métodos descendentes recursivos y construye el AST. Si encuentra errores sintácticos, utiliza recuperación en modo pánico para continuar el análisis.

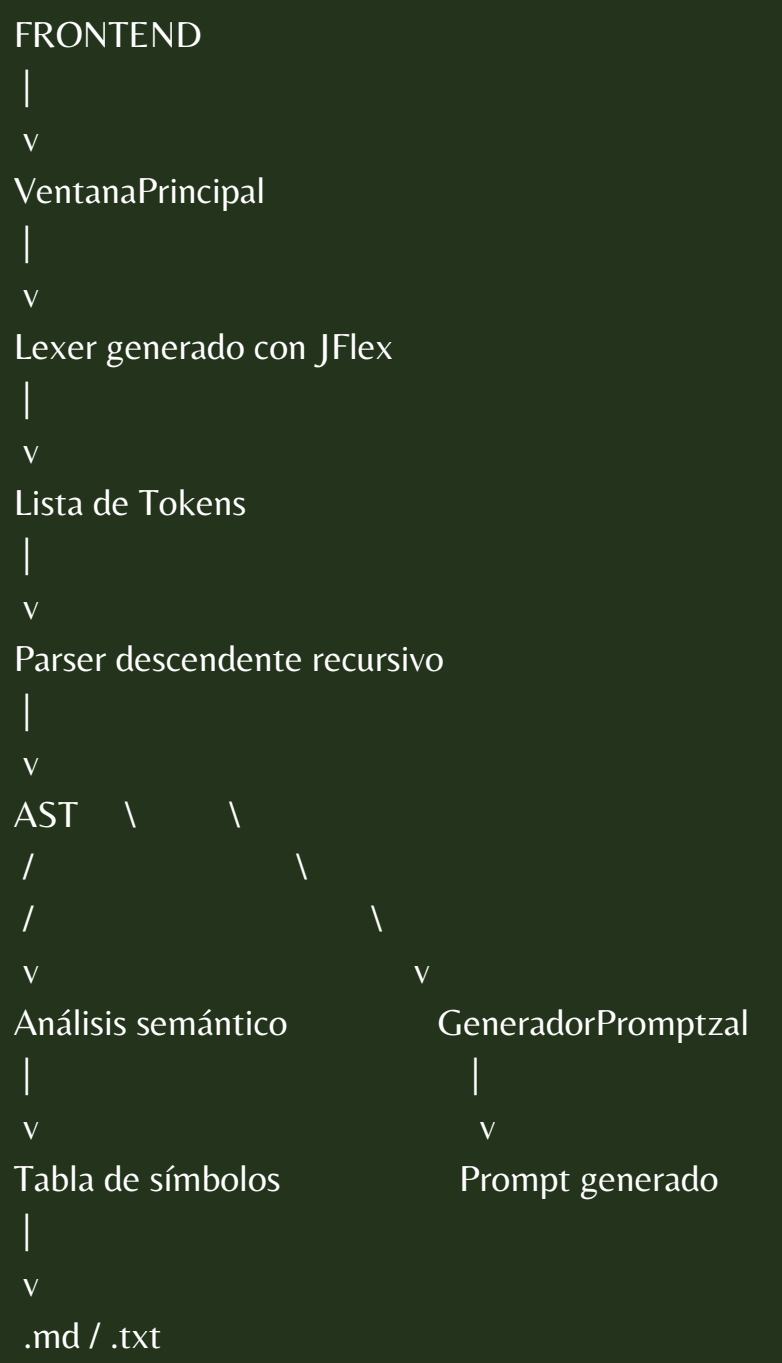
El AST generado se utiliza posteriormente en dos procesos principales. El primero es el análisis semántico, encargado de validar el significado del programa y construir la tabla de símbolos. El segundo es la generación del promptzal final.

Una característica fundamental de esta arquitectura es que el generador del prompt no procesa nuevamente el texto original del editor. El método de generación recibe directamente el objeto Programa que representa la raíz del AST y recorre sus nodos para construir el resultado final.

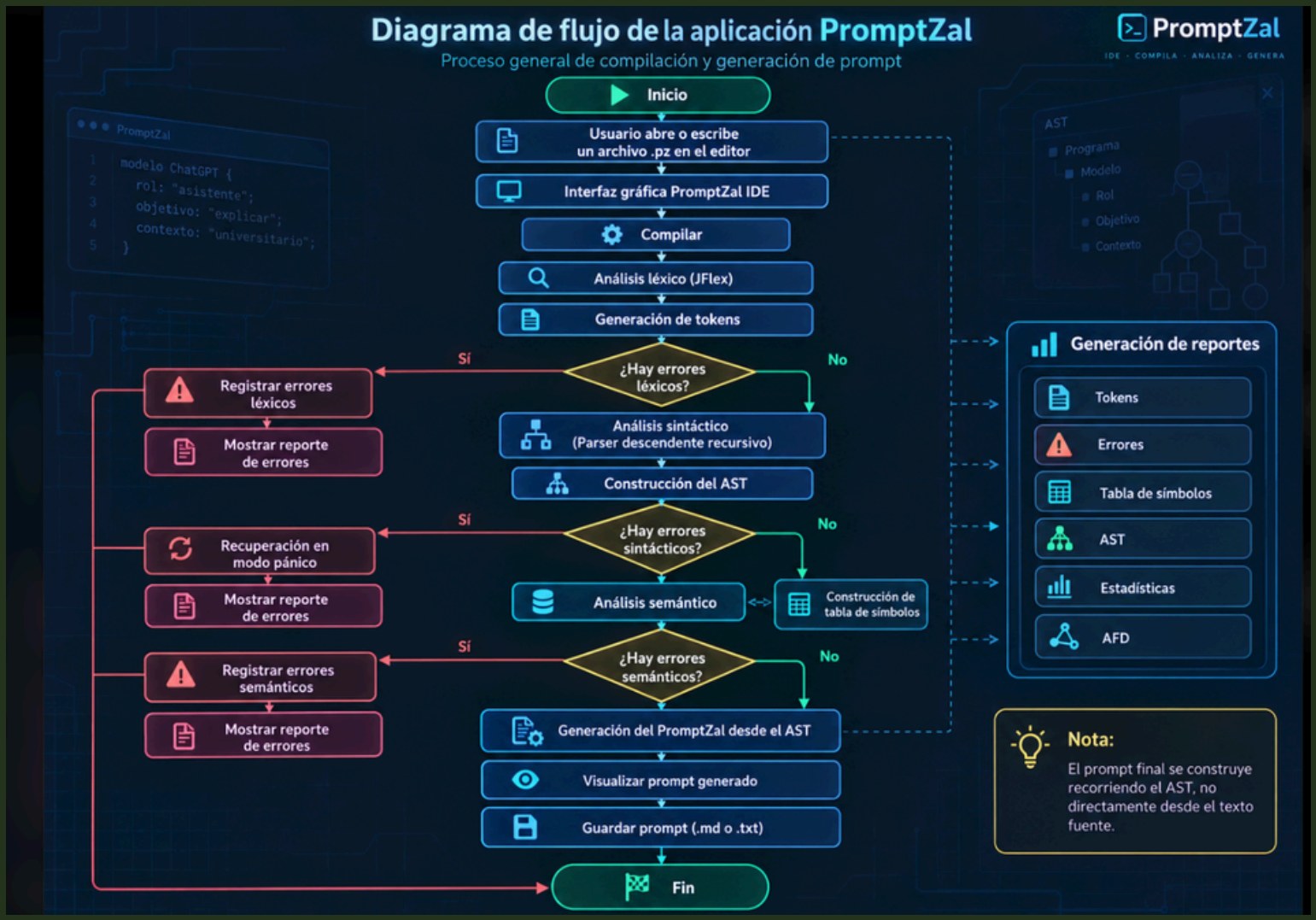
La arquitectura general puede representarse de la siguiente forma:



La arquitectura general puede representarse de la siguiente forma:



# flujo general de aplicacion



## 2. ESTRUCTURA GENERAL DEL PROYECTO

La estructura del código fuente se organiza principalmente bajo el paquete:

com.practica.promptzal

La división general utilizada es la siguiente:

```
com.practica.promptzal
|
|-- PromptZal.java
|
|-- Backend
|    |
|    |-- afd
|    |
|    |-- archivos
|    |
|    |-- ast
|    |
|    |-- generador
|    |
|    |-- lexer
|    |
|    |-- parser
|    |
|    |-- reporte
|     |       |
|     |       |-- estiloReporte
|     |
|     |-- semantico
|
|-- Frontend
```

# 3. ARQUITECTURA BACKEND Y FRONTEND

## 3.1 Backend

El Backend agrupa todas las operaciones relacionadas con el procesamiento del lenguaje PromptZal. Ninguna de estas clases depende directamente de botones, ventanas o componentes visuales para realizar el análisis.

Sus principales responsabilidades son:

- Generar tokens mediante JFlex.
- Registrar errores léxicos.
- Analizar la estructura sintáctica.
- Construir el AST.
- Recuperarse de errores sintácticos.
- Construir la tabla de símbolos.
- Ejecutar las validaciones semánticas.
- Generar el promptzal.
- Generar los reportes.
- Crear las representaciones gráficas del AFD y del AST.

La separación del Backend permite que el mismo proceso de compilación pueda ejecutarse independientemente de la forma en que el resultado sea presentado al usuario.

## 3.2 Frontend

El Frontend fue desarrollado utilizando Java Swing. Su función es coordinar las acciones realizadas por el usuario y presentar los resultados producidos por el Backend.

Las principales clases de esta capa son:

ventanaPrincipal:
Es la ventana principal de la aplicación. Coordina las acciones de crear, abrir, guardar y compilar archivos, además de solicitar la generación de reportes.

GestorVistaIDE:
Administra la organización visual del entorno. Controla la aparición y ocultamiento del explorador, panel de prompt y panel de resultados.

EstiloIDE:
Centraliza colores, fuentes, bordes, estilos de botones, tablas y demás elementos visuales de la aplicación.
TablaTokens:
Muestra los tokens reconocidos durante la última compilación.

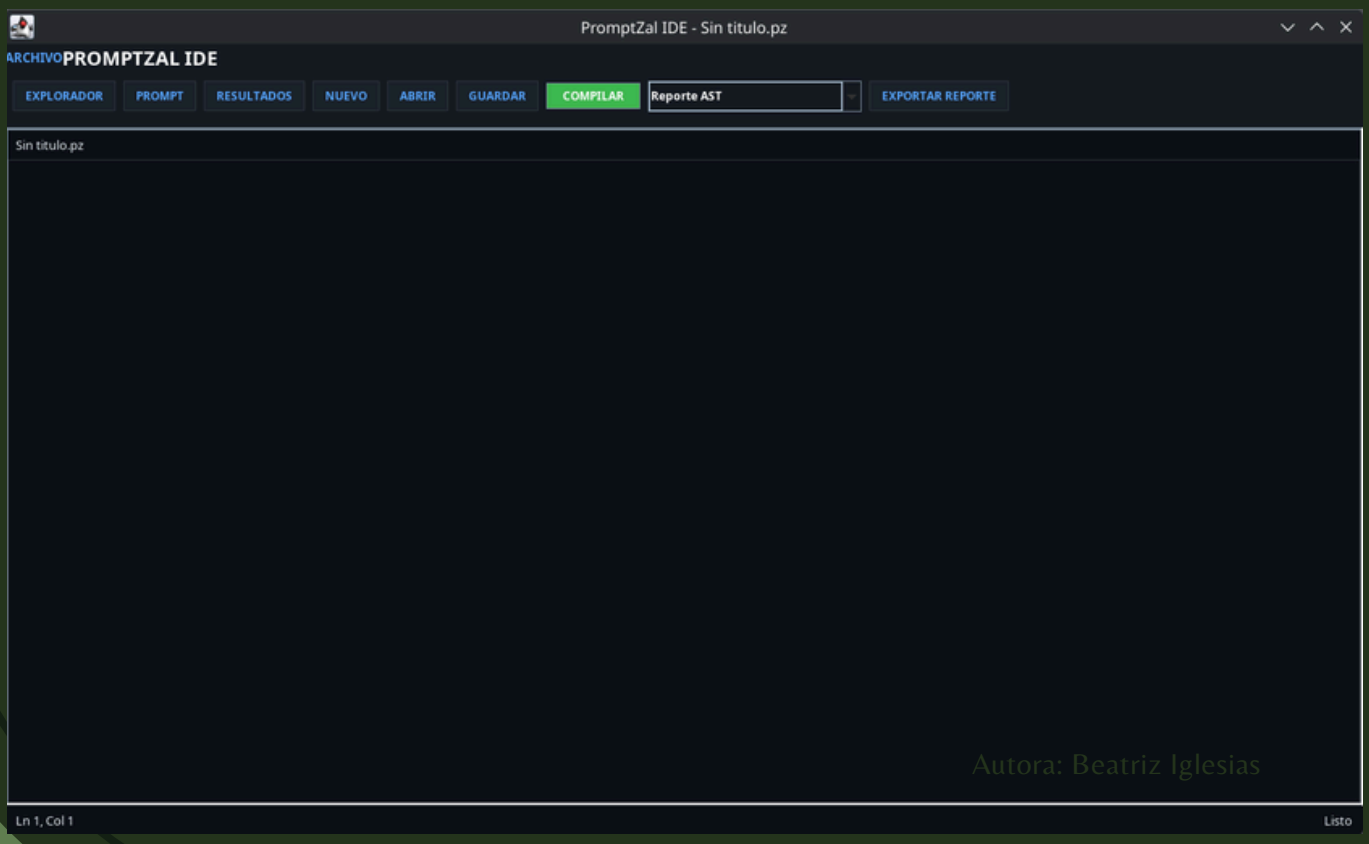
TablaErroes:
Muestra errores léxicos, sintácticos y semánticos.

TablaEstadisticas:
Muestra información estadística de la compilación.

TablaSimbolos:
Muestra los símbolos registrados durante el análisis semántico.

TablaSalida:
Muestra mensajes relacionados con las diferentes fases de compilación.

La interfaz fue diseñada para mantener el editor como elemento principal. Los paneles de explorador, prompt y resultados pueden mostrarse u ocultarse según las necesidades del usuario, recuperando automáticamente el espacio disponible para el editor.



# 4. FLUJO GENERAL DEL COMPILADOR

El compilador PromptZal se encuentra dividido en varias fases que se ejecutan de manera secuencial. Cada fase recibe la salida de la etapa anterior, realiza una responsabilidad específica y genera información que será utilizada por las siguientes etapas.

El proceso inicia cuando el usuario escribe directamente en el editor de la aplicación o abre un archivo con extensión .pz. Al presionar el botón Compilar, el contenido del editor es enviado al analizador léxico generado mediante JFlex.

La secuencia general es la siguiente:

1. Entrada del programa PromptZal.
2. Análisis léxico con JFlex.
3. Generación de tokens y registro de errores léxicos.
4. Análisis sintáctico mediante parser descendente recursivo.
5. Construcción simultánea del Árbol de Sintaxis Abstracta.
6. Recuperación de errores sintácticos mediante modo pánico.
7. Análisis semántico.
8. Construcción de la tabla de símbolos.
9. Generación del promptzal recorriendo el AST.
10. Presentación del resultado en la interfaz.
11. Posibilidad de guardar el prompt generado como .md o .txt.
12. Generación de reportes.

El analizador léxico transforma el texto en una lista de objetos Token. Cada token contiene el tipo, el lexema, la fila y la columna donde inicia. Los comentarios, espacios, tabulaciones y saltos de línea no se envían al parser.

Posteriormente, la lista de tokens es procesada por la clase Parser. Esta clase implementa manualmente la gramática libre de contexto mediante métodos descendentes recursivos. Mientras reconoce las estructuras válidas, crea los objetos que conforman el AST.

El análisis semántico recibe el AST y verifica las reglas de significado del lenguaje. Durante esta etapa también se construye la tabla de símbolos.

Finalmente, si la compilación es válida, GeneradorPromptzal recibe el objeto Programa, que representa la raíz del AST, y genera el prompt final. Esta decisión evita reconstruir información a partir del texto original y mantiene una separación clara entre análisis y generación.

## 5. DISEÑO LÉXICO FORMAL

El análisis léxico constituye la primera fase del compilador. Su objetivo es transformar la secuencia de caracteres escrita por el usuario en unidades mínimas con significado denominadas tokens.

PromptZal conserva el diseño léxico formal desarrollado en las etapas anteriores del proyecto. Dicho diseño define las categorías que pueden aparecer en el lenguaje y sirve como base conceptual para la especificación implementada posteriormente mediante JFlex.

Las principales categorías léxicas son:

- Directivas.
- Palabras reservadas de estructura.
- Comandos de inteligencia artificial.
- Función del sistema CARGAR.
- Conectores.
- Identificadores.
- Literales de cadena.
- Números enteros y decimales.
- Operadores.
- Delimitadores.
- Comentarios.
- Fin de archivo.

# 6. AUTÓMATA FINITO DETERMINISTA

## 6.1 Propósito del AFD

El Autómata Finito Determinista representa formalmente las formas léxicas reconocidas por PromptZal. Su objetivo es mostrar los estados y transiciones necesarios para reconocer identificadores, números, cadenas, directivas, operadores, delimitadores y comentarios.

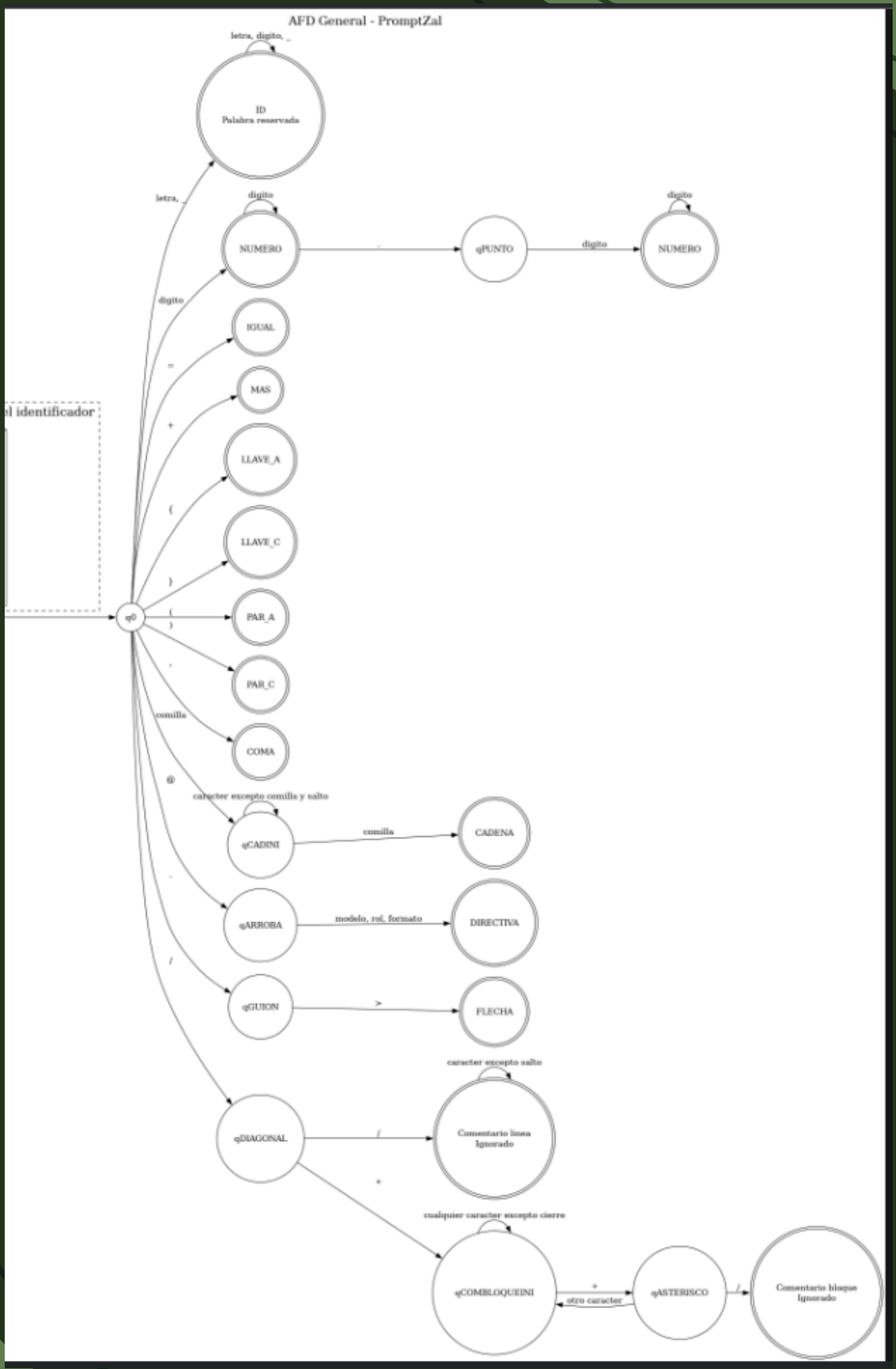
En la implementación actual, la clase GeneradorAFD construye un archivo DOT que describe un AFD general del lenguaje. Posteriormente Graphviz procesa el archivo y produce una imagen PNG.

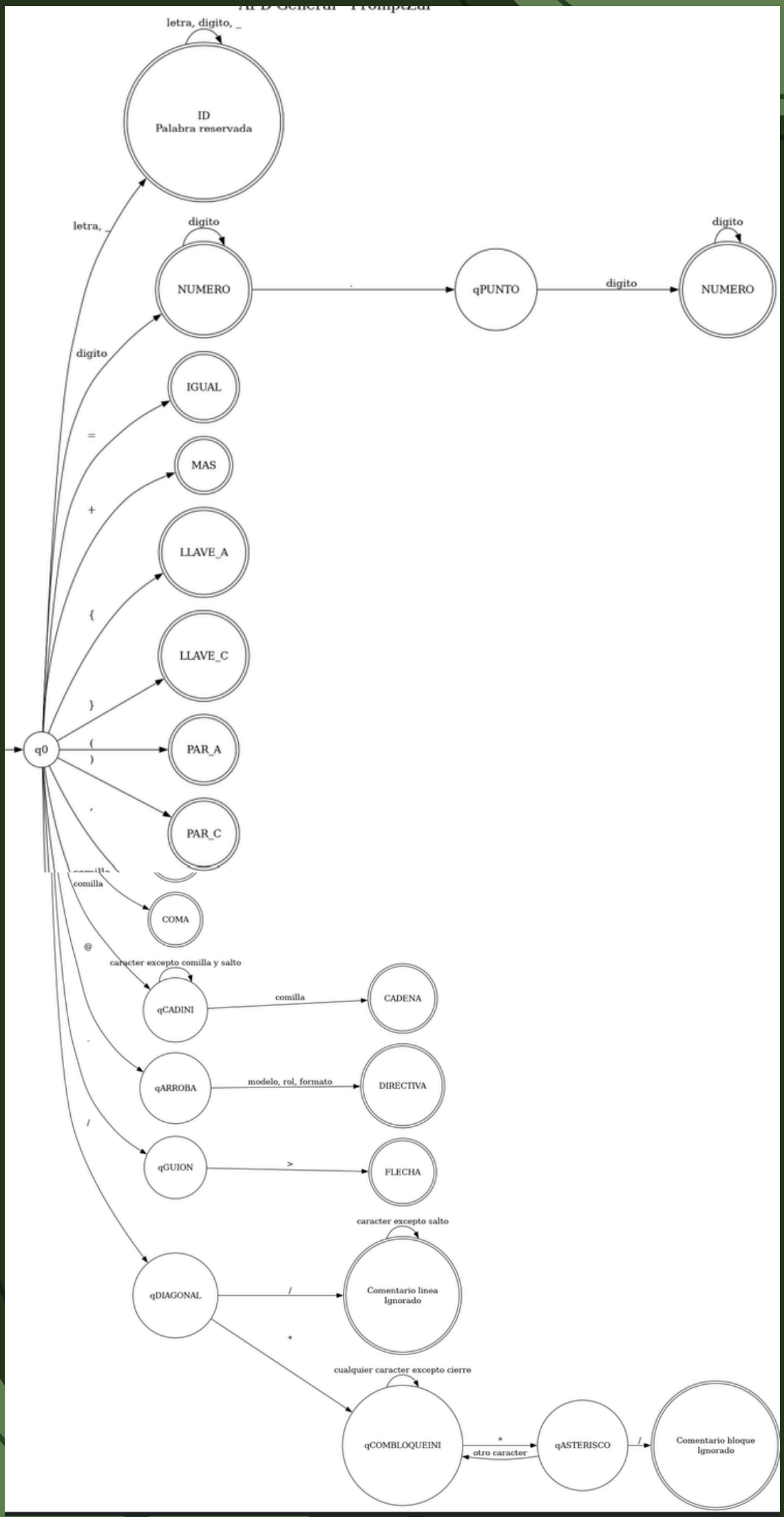
El AFD generado es una representación general y no depende del programa .pz que se encuentre abierto. Por esta razón siempre representa el conjunto completo de categorías léxicas soportadas por PromptZal.

Figura X. Autómata Finito Determinista general de PromptZal.

## 6.2 Estado inicial

El recorrido comienza en el estado q0. Desde este estado se determina la categoría léxica de acuerdo con el primer carácter leído.


AFD General - PromptZal
letra, digito, _
ID
Palabra reservada
letra, _
digito
NUMERO
.
qPUNTO
digito
digito
NUMERO
digito
IGUAL
=
MAS
+
LLAVE_A
{
LLAVE_C
}
(
)
PAR_A
PAR_C
,
comilla
COMA
@
caracter excepto comilla y salto
qCADINI
comilla
CADENA
qARROBA
modelo, rol, formato
DIRECTIVA
-
qGUION
>
FLECHA
/
qDIAGONAL
/
Comentario linea
Ignorado
caracter excepto salto
*
qCOMBLOQUEINI
cualquier caracter excepto cierre
*
otro caracter
qASTERISCO
/
Comentario bloque
Ignorado


letra, digito, _
ID
Palabra reservada
letra, _
digito
NUMERO
.
qPUNTO
digito
NUMERO
digito
digito
IGUAL
=
MAS
+
LLAVE_A
{
LLAVE_C
}
q0
(
)
PAR_A
PAR_C
,
comilla
COMA
@
caracter excepto comilla y salto
qCADINI
comilla
CADENA
.
qARROBA
modelo, rol, formato
DIRECTIVA
/
qGUION
>
FLECHA
caracter excepto salto
qDIAGONAL
/
Comentario linea
Ignorado
*
cualquier caracter excepto cierre
qCOMBLOQUEINI
*
otro caracter
qASTERISCO
/
Comentario bloque
Ignorado

Las transiciones principales son:

q0 → qID
Cuando se encuentra una letra o guion bajo.

q0 → qNUM
Cuando se encuentra un dígito.

q0 → qCADINI
Cuando se encuentra una comilla doble.

q0 → qARROBA
Cuando se encuentra el símbolo @.

q0 → qIGUAL
Cuando se encuentra =.

q0 → qMAS
Cuando se encuentra +.

q0 → qGUION
Cuando se encuentra -.

q0 → qLLAVEA
Cuando se encuentra {.

q0 → qLLAVEC
Cuando se encuentra }.

q0 → qPARA
Cuando se encuentra (.

q0 → qPARC
Cuando se encuentra ).

q0 → qCOMA
Cuando se encuentra ,.

q0 → qDIAGONAL
Cuando se encuentra /.

## 6.3 Generación del AFD con Graphviz

La clase:

Backend.afd.GeneradorAFD

construye la representación DOT del autómata.

Proceso:

GeneradorAFD
|
v
construirDot()
|
v
afd_promptzal.dot
|
v
Graphviz: dot -Tpng
|
v
afd_promptzal.png

```
private String construirDot() {
    StringBuilder dot = new StringBuilder();

    dot.append("digraph AFD_PromptZal {\n");
    dot.append("rankdir=LR;\n");
    dot.append("labelloc=\"t\";\n");
    dot.append("label=\"AFD General - PromptZal\";\n");
    dot.append("fontsize=20;\n");
    dot.append("node [shape=circle];\n\n");

    // Estado inicial
    dot.append("inicio [shape=point];\n");
    dot.append("q0 [label=\"q0\"];\n");
    dot.append("inicio -> q0;\n\n");

    // Estados de aceptacion
    dot.append("qID [shape=doublecircle, label=\"ID\\nPalabra reservada\"];\n");
    dot.append("qNUM [shape=doublecircle, label=\"NUMERO\"];\n");
    dot.append("qCAD [shape=doublecircle, label=\"CADENA\"];\n");
    dot.append("qDIR [shape=doublecircle, label=\"DIRECTIVA\"];\n");
    dot.append("qIGUAL [shape=doublecircle, label=\"IGUAL\"];\n");
    dot.append("qMAS [shape=doublecircle, label=\"MAS\"];\n");
    dot.append("qFLECHA [shape=doublecircle, label=\"FLECHA\"];\n");
    dot.append("qLLAVEA [shape=doublecircle, label=\"LLAVE_A\"];\n");
    dot.append("qLLAVEC [shape=doublecircle, label=\"LLAVE_C\"];\n");
    dot.append("qPARA [shape=doublecircle, label=\"PAR_A\"];\n");
    dot.append("qPARC [shape=doublecircle, label=\"PAR_C\"];\n");
    dot.append("qCOMA [shape=doublecircle, label=\"COMA\"];\n");
    dot.append("qCOMLINEA [shape=doublecircle, label=\"Comentario linea\\nIgnorado\"];\n");
    dot.append("qCOMBLOQUE [shape=doublecircle, label=\"Comentario bloque\\nIgnorado\"];\n\n");
```

```java
private boolean generarImagen() {
    try {
        ProcessBuilder proceso = new ProcessBuilder(
                "dot",
                "-Tpng",
                rutaDot,
                "-o",
                rutaImagen
        );

        Process ejecucion = proceso.start();

        int resultado = ejecucion.waitFor();

        if (resultado == 0) {
            return true;
        }

        System.out.println( "Graphviz no pudo generar la imagen del AFD." );

        return false;

    } catch (IOException e) {
        System.out.println("No se pudo ejecutar Graphviz: "+ e.getMessage());

        return false;

    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();

        System.out.println(
                "La generacion del AFD fue interrumpida."
```

# 7. GRAMÁTICA REGULAR DEL COMPONENTE LÉXICO

La gramática regular describe formalmente el mismo lenguaje reconocido por el Autómata Finito Determinista de PromptZal.

Se define como:

G = {N, T, P, S}

N = {
  q0,
  qID,
  qNUM,
  qPUNTO,
  qDEC,
  qCADINI,
  qCAD,
  qARROBA,
  qDIR,
  qIGUAL,
  qMAS,
  qGUION,
  qFLECHA,
  qLLAVEA,
  qLLAVEC,
  qPARA,
  qPARC,
  qCOMA,
  qDIAGONAL,
  qCOMLINEA,
  qCOMBLOQUEINI,
  qASTERISCO,
  qCOMBLOQUE
}

T = {
letra,
digito,
"_",
".",
comilla,
"@",
"modelo",
"rol",
"formato",
"=",
"+",
"-",
">",
"{",
"}",
"(",
")",
",",
"/",
"*",
caracter_excepto_comilla_y_salto,
caracter_excepto_salto,
cualquier_caracter_excepto_cierre,
otro_caracter
}

S = q0

8.3 Macros léxicas

Identificador:

[a-zA-Z_][a-zA-Z0-9_]*

Número:

[0-9]+(\.[0-9]+)?

```
%}

     // Expresiones

     ESPACIO = [ \t\f\r\n]+
    IDENTIFICADOR = [a-zA-Z_][a-zA-Z0-9_]*
    NUMERO = [0-9]+(\.[0-9]+)?
    CADENA = \"[^\r\n\"]*\"
    CADENA_SIN_CERRAR = \"[^\r\n\"]*
    DIRECTIVA = "@modelo"|"@rol"|"@formato"
    DIRECTIVA_DESCONOCIDA = @[a-zA-Z_][a-zA-Z0-9_]*
    COMANDO_IA = "PREGUNTAR"|"GENERAR"|"RESUMIR"|"ANALIZAR"|"TRADUCIR"|"CLASIFICAR"|"EXTRAER"
    CONECTOR = "SOBRE"|"DESDE"|"EN"|"COMO"
    COMENTARIO_LINEA = "//"[^\r\n]*

%%
```

8.4 Directivas

@modelo
@rol
@formato

→ DIRECTIVA

8.5 Palabras reservadas

AGENTE → AGENTE
contexto → CONTEXTO
variable → VARIABLE
EJECUTAR → EJECUTAR
EXPORTAR → EXPORTAR

8.6 Comandos

PREGUNTAR
GENERAR
RESUMIR
ANALIZAR
TRADUCIR
CLASIFICAR
EXTRAER

→ COMANDO_IA

```
// Palabras de estructura

"AGENTE" {
    return crearToken(TipoToken.AGENTE);
}

"contexto" {
    return crearToken(TipoToken.CONTEXTO);
}

"variable" {
    return crearToken(TipoToken.VARIABLE);
}

"EJECUTAR" {
    return crearToken(TipoToken.EJECUTAR);
}

"EXPORTAR" {
    return crearToken(TipoToken.EXPORTAR);
}

// Comandos IA

{COMANDO_IA} {
    return crearToken(TipoToken.COMANDO_IA);
}
```

8.7 Identificadores

Los identificadores no reservados producen ID.

8.8 Cadenas

Las cadenas válidas producen CADENA.

Las cadenas sin cerrar generan CADENA_SIN_CERRAR.

8.9 Números

Los enteros y decimales producen NUMERO.

8.10 Comentarios y espacios

Se descartan y no llegan al parser.

8. 11 cErrores léxicos

CARACTER_NO_RECONOCIDO
CADENA_SIN_CERRAR
DIRECTIVA_NO_RECONOCIDA
COMENTARIO_BLOQUE_SIN_CERRAR

```
<COMENTARIO_BLOQUE><<EOF>> {
    agregarError(
        comentarioBloque.toString(),
                TipoErrorLexico.COMENTARIO_BLOQUE_SIN_CERRAR,
                filaComentario,
                columnaComentario
    );

    yybegin(YYINITIAL);
    return crearEOF();
}
```

```
{DIRECTIVA_DESCONOCIDA} {
    agregarError(
        yytext(),
                TipoErrorLexico.DIRECTIVA_NO_RECONOCIDA,
                yyline + 1,
                yycolumn + 1
    );
}
```

```
 {CADENA_SIN_CERRAR} {
     agregarError(
         yytext(),
                 TipoErrorLexico.CADENA_SIN_CERRAR,
                 yyline + 1,
                 yycolumn + 1
     );
 }
```

```
// Caracter invalido

. {
    agregarError(yytext(),
                TipoErrorLexico.CARACTER_NO_RECONOCIDO,
                yyline + 1,
                yycolumn + 1
    );
}
```

# 9. TABLA DE TERMINALES

| B | C |
|---|---|
| DIRECTIVA | @modelo, @rol, @formato |
| AGENTE | AGENTE |
| CONTEXTO | contexto |
| VARIABLE | variable |
| EJECUTAR | EJECUTAR |
| EXPORTAR | EXPORTAR |
| COMANDO_IA | PREGUNTAR, GENERAR, RESUMIR, ANALIZAR, TRADUCIR, CLASIFICAR, EXTRAER |
| CARGAR | CARGAR |
| CONECTOR | SOBRE, DESDE, EN, COMO |
| FLECHA | → |
| IGUAL | = |
| MAS | + |
| LLAVE_A | { |
| LLAVE_C | } |
| PAR_A | ( |
| PAR_C | ) |
| COMA | , |
| ID | Identificadores |
| CADENA | Texto entre comillas dobles |
| NUMERO | Enteros y decimales |
| EOF | Fin de archivo |
| | |

```
public enum TipoToken {

    DIRECTIVA("Directiva"),
    AGENTE("Agente"),
    CONTEXTO("Contexto"),
    VARIABLE("Variable"),
    EJECUTAR("Ejecutar"),
    EXPORTAR("Exportar"),
    COMANDO_IA("Comando de IA"),
    CARGAR("Funcion del sistema"),
    CONECTOR("Conector"),
    FLECHA("Flecha"),
    IGUAL("Asignacion"),
    MAS("Concatenacion"),
    LLAVE_A("Llave de apertura"),
    LLAVE_C("Llave de cierre"),
    PAR_A("Parentesis de apertura"),
    PAR_C("Parentesis de cierre"),
    COMA("Coma"),
    ID("Identificador"),
    CADENA("Literal de cadena"),
    NUMERO("Literal numerico"),
    EOF("Fin de archivo");
```

## 10. GRAMÁTICA LIBRE DE CONTEXTO EN BNF

<programa> -> <directivas> <agentes> <ejecuciones> <exportar> EOF

<directivas> -> DIRECTIVA CADENA <directivas> | ε

<agentes> -> <agente> <mas_agentes>

<mas_agentes> -> <agente> <mas_agentes> | ε

<agente> -> AGENTE ID LLAVE_A <contexto> <variables> <comandos> LLAVE_C

<contexto> -> CONTEXTO IGUAL CADENA

<variables> -> VARIABLE ID IGUAL <expresion> <variables> | ε

<comandos> -> <comando> <mas_comandos>

<mas_comandos> -> <comando> <mas_comandos> | ε

<comando> -> COMANDO_IA <expresion> <conectores> FLECHA ID

<conectores> -> CONECTOR <expresion> <conectores> | ε

<expresion> -> <termino> <mas_terminos>

<mas_terminos> -> MAS <termino> <mas_terminos> | ε

<termino> -> CADENA | ID | NUMERO <unidad> | <cargar>

<unidad> -> ID | ε

<cargar> -> CARGAR PAR_A CADENA PAR_C

<ejecuciones> -> EJECUTAR ID <mas_ejecuciones>

<mas_ejecuciones> -> EJECUTAR ID <mas_ejecuciones> | ε

<exportar> -> EXPORTAR ID <mas_ids>

<mas_ids> -> COMA ID <mas_ids> | ε

```java
// <programa> -> <directivas> <agentes> <ejecuciones> <exportar> EOF
private Programa programa() {
    Programa programa = new Programa();

    directivas(programa);
    agentes(programa);
    ejecuciones(programa);
    exportar(programa);
    consumir(TipoToken.EOF, "EOF");

    return programa;
}

// <directivas> -> DIRECTIVA CADENA <directivas> | epsilon
private void directivas(Programa programa) {
    if (!verificar(TipoToken.DIRECTIVA)) {
        return;
    }

    Token directiva = avanzar();

    Token valor = consumir(
            TipoToken.CADENA,
            "CADENA",
            TipoToken.DIRECTIVA,
            TipoToken.AGENTE
    );
```

```java
// <mas_agentes> -> <agente> <mas_agentes> | epsilon
private void masAgentes(Programa programa) {
    if (!verificar(TipoToken.AGENTE)) {
        return;
    }

    Agente agente = agente();

    if (agente != null) {
        programa.agregarAgente(agente);
    }

    masAgentes(programa);
}

// <agente> -> AGENTE ID LLAVE_A <contexto> <variables> <comandos> LLA
private Agente agente() {
    Token inicio = consumir(
            TipoToken.AGENTE,
            "AGENTE"
    );

    Token nombre = consumir(
            TipoToken.ID,
            "ID",
            TipoToken.LLAVE_A,
            TipoToken.CONTEXTO
    );
```

## 11. PARSER DESCENDENTE RECURSIVO

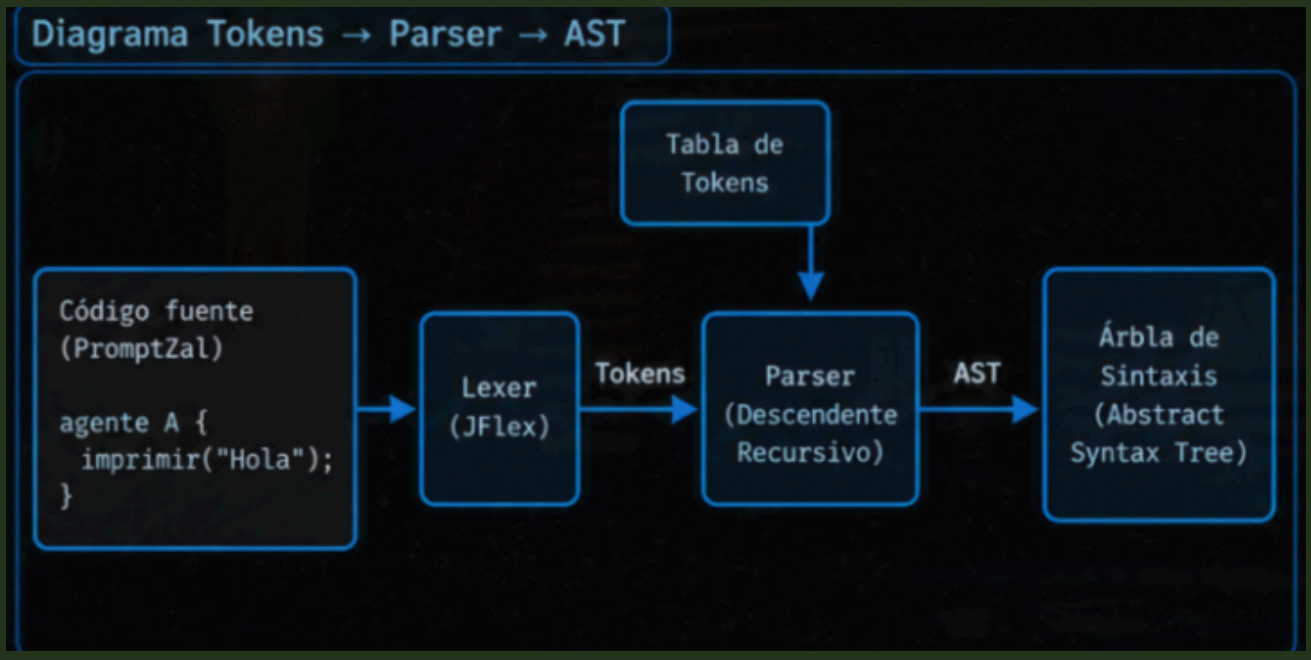
El analizador sintáctico de PromptZal está implementado manualmente en:

com.practica.promptzal.Backend.parser.Parser

El parser recibe la lista de tokens generada por JFlex y verifica que cumpla la gramática BNF.

Flujo:

Tokens
  |
  v
Parser
  |
  +----> Errores sintácticos
  |
  v
AST



11.1 Método parsear()

Inicia el análisis, limpia errores anteriores y llama a programa().

```
public Programa parsear() {
    posicion = 0;
    errores.clear();
    return programa();
}

public ArrayList<ErrorSintactico> getErrores() {
    return errores;
}
```

11.2 Método programa()

Representa:

<programa> -> <directivas> <agentes> <ejecuciones> <exportar> EOF

Crea la raíz Programa y coordina el resto del análisis.

```
private Programa programa() {
    Programa programa = new Programa();

    directivas(programa);
    agentes(programa);
    ejecuciones(programa);
    exportar(programa);
    consumir(TipoToken.EOF, "EOF");

    return programa;
}
```

11.3 Método agente()

Reconoce la estructura de un agente y crea el objeto Agente correspondiente.

```java
private void agentes(Programa programa) {
    if (!verificar(TipoToken.AGENTE)) {
        registrarError("AGENTE");

        sincronizar(
                TipoToken.AGENTE,
                TipoToken.EJECUTAR,
                TipoToken.EXPORTAR
        );
    }

    if (verificar(TipoToken.AGENTE)) {
        Agente agente = agente();

        if (agente != null) {
            programa.agregarAgente(agente);
        }

        masAgentes(programa);
    }
}
```

11.4 Método comando()

Reconoce:

COMANDO_IA expresion conectores -> ID

y construye el objeto Comando.

```java
// <comando> -> COMANDO_IA <expresion> <conectores> FLECHA ID
private Comando comando() {
    Token tipo = consumir(TipoToken.COMANDO_IA,"COMANDO_IA" );

    Expresiones expresionPrincipal = expresion();

    ArrayList<ConectorComando> listaConectores = conectores();

    consumir(
            TipoToken.FLECHA,
            "FLECHA",
            TipoToken.ID,
            TipoToken.COMANDO_IA,
            TipoToken.LLAVE_C
    );

    Token destino = consumir(
            TipoToken.ID,
            "ID",
            TipoToken.COMANDO_IA,
            TipoToken.LLAVE_C,
            TipoToken.EJECUTAR,
            TipoToken.EXPORTAR
    );
```

12. CORRESPONDENCIA GRAMÁTICA – MÉTODOS

| NO TERMINALES | METODOS |
|---|---|
| <programa> | programa() |
| <directivas> | directivas() |
| <agentes> | agentes() |
| <mas_agentes> | masAgentes() |
| <agente> | agente() |
| <contexto> | contexto() |
| <variables> | variables() |
| <comandos> | comandos() |
| <mas_comandos> | masComandos() |
| <comando> | comando() |
| <conectores> | conectores() |
| <expresion> | expresion() |
| <mas_terminos> | masTerminos() |
| <termino> | termino() |
| <unidad> | unidad() |
| <cargar> | cargar() |
| <ejecuciones> | ejecuciones() |
| <mas_ejecuciones> | masEjecuciones() |
| <exportar> | exportar() |
| <mas_ids> | masIds() |

13. ÁRBOL DE SINTAXIS ABSTRACTA

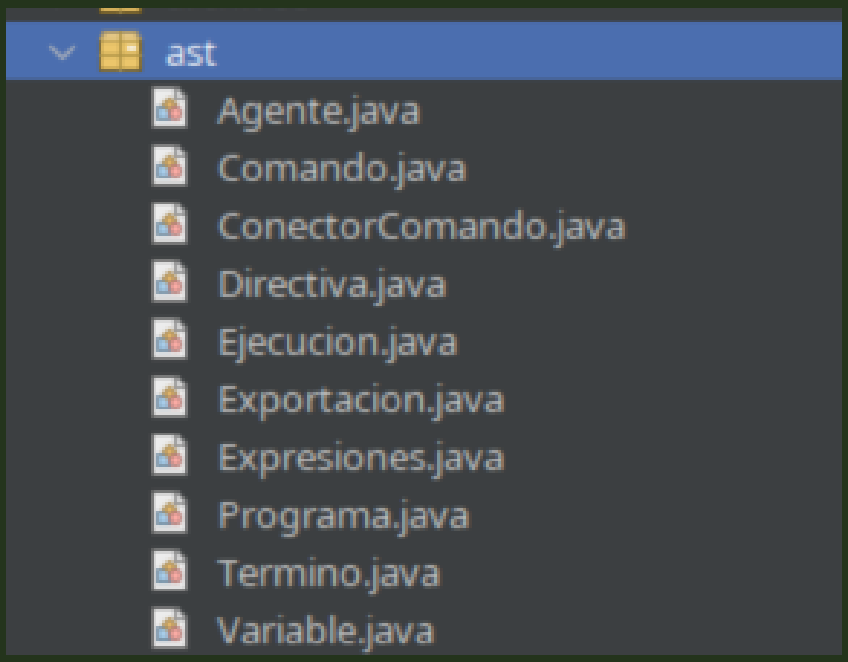
El AST representa de forma estructurada el programa reconocido por el parser.

La raíz es Programa y desde ella se organizan las directivas, agentes, ejecuciones y exportaciones.

Cada agente contiene su contexto, variables y comandos.

El árbol se construye directamente durante el proceso de parsing.

Programa
Directiva
Agente
Variable
Expresiones
Termino
Comando
ConectorComando
Ejecucion
Exportacion

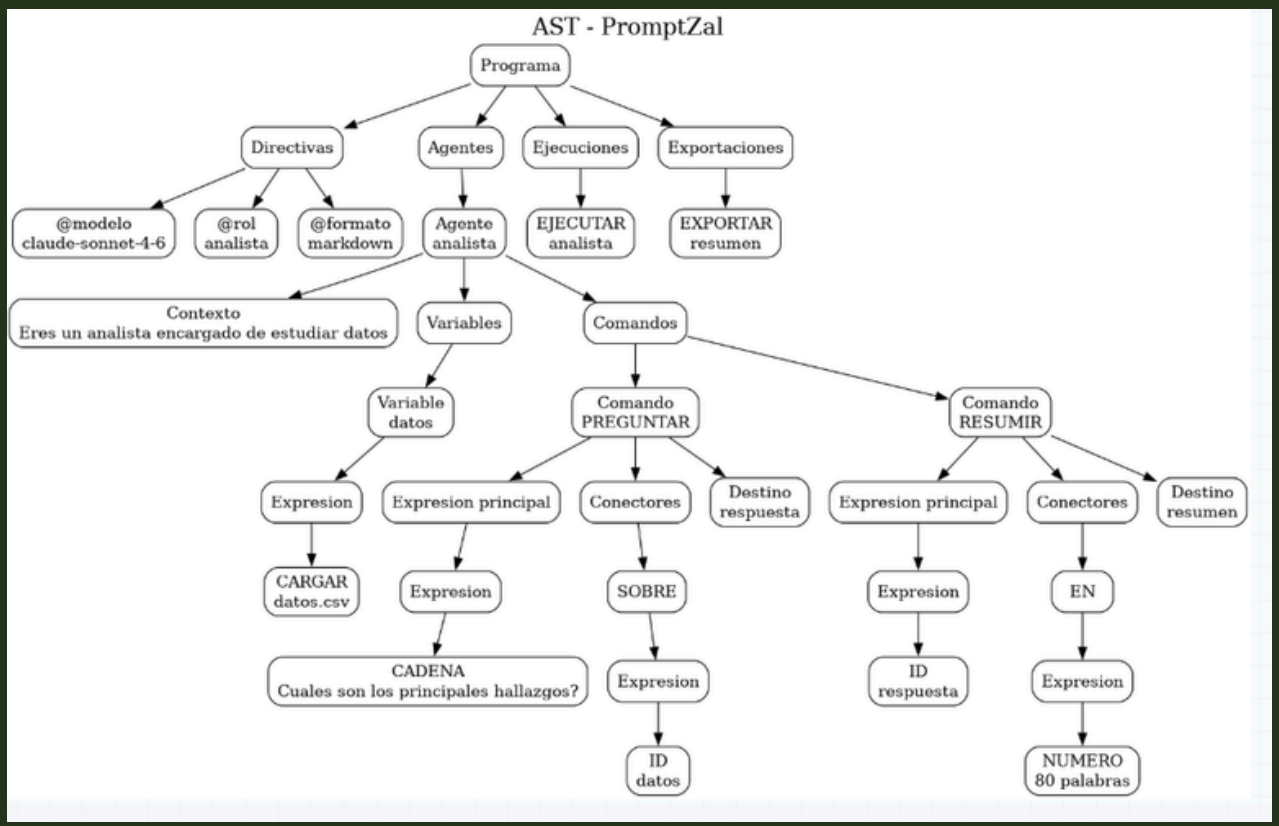


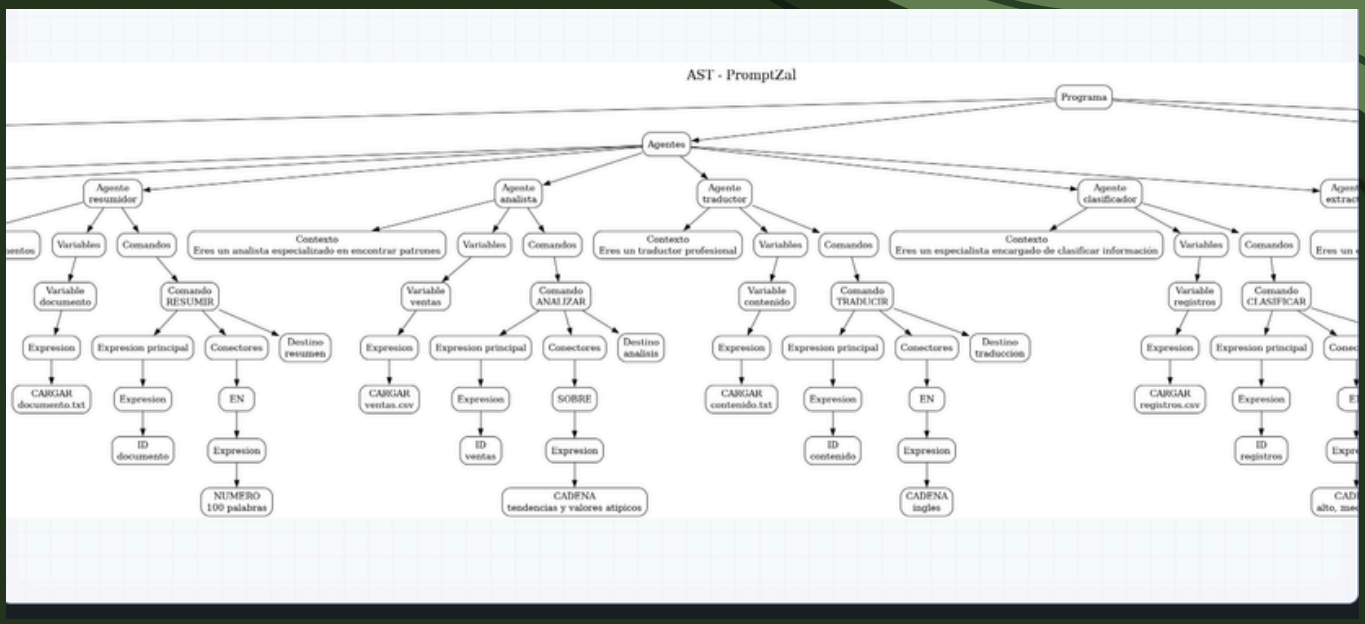
14. DIAGRAMA DE CLASES DEL AST

Relaciones principales:

Programa
|
|-- Directiva
|-- Agente
|  |
|  |-- Variable
|  |  └──── Expresiones
|  |  └──── Termino
|  |
|  └──── Comando
|  |-- Expresiones
|  └──── ConectorComando
|  └──── Expresiones
|
|-- Ejecucion
└──── Exportacion

15. AST DINÁMICO






16 TABLA DE SÍMBOLOS

La tabla de símbolos almacena los elementos declarados durante el análisis semántico.

Las clases principales son:

Backend.semantico.Simbolo
Backend.semantico.TablaSimbolos

Cada símbolo registra:

- nombre
- tipo
- ámbito
- línea

Los tipos utilizados son:

AGENTE
VARIABLE_DECLARADA
RESULTADO_COMANDO

Los agentes utilizan el ámbito GLOBAL.

Las variables declaradas y los resultados de comandos utilizan como ámbito el nombre del agente al que pertenecen.

```java
public class Simbolo {

    private String nombre;
    private String tipo;
    private String ambito;
    private int linea;

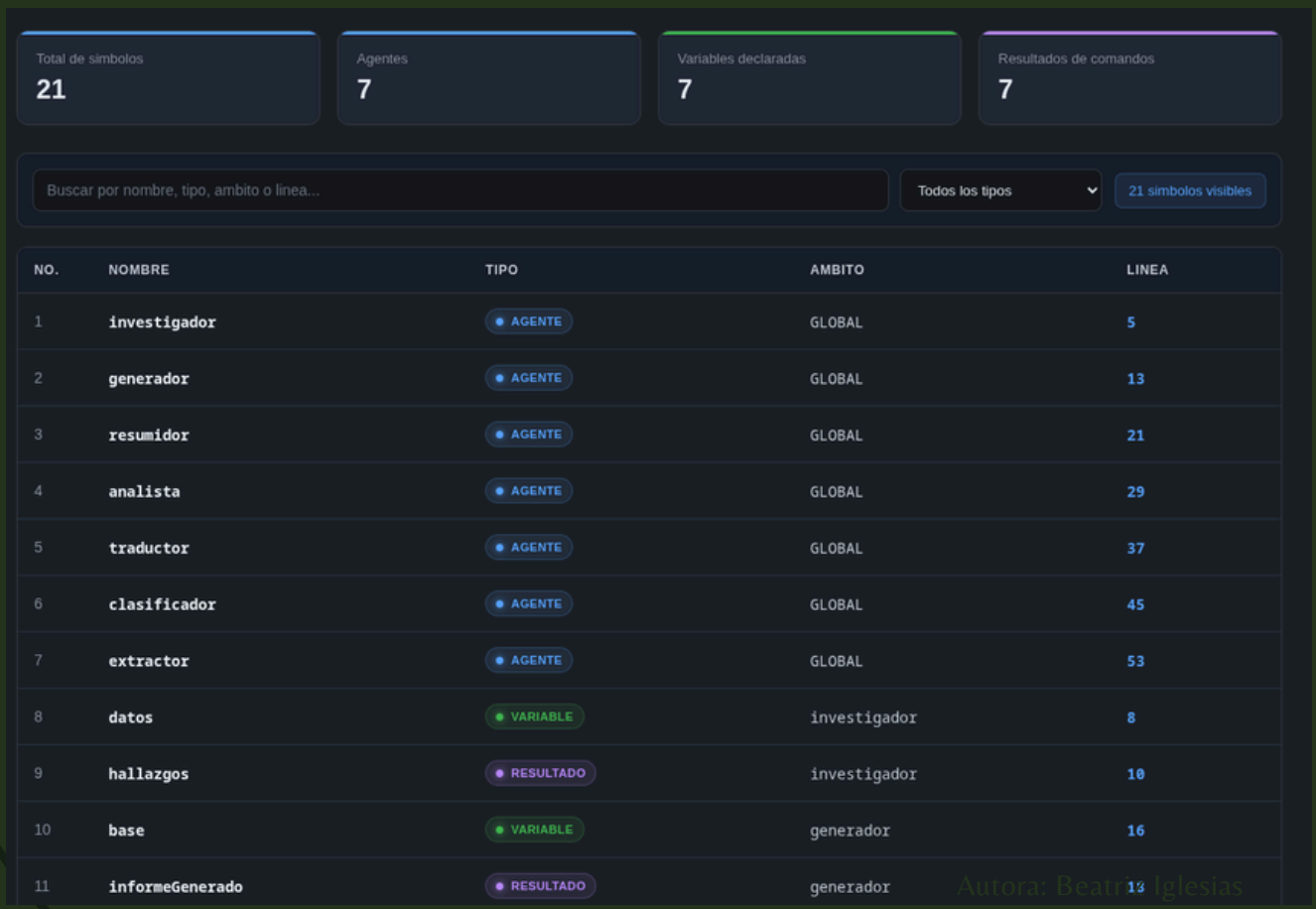
    public Simbolo(String nombre, String tipo, String ambito, int linea) {
        this.nombre = nombre;
        this.tipo = tipo;
        this.ambito = ambito;
        this.linea = linea;
    }

    public String getNombre() {
        return nombre;
    }

    public String getTipo() {
        return tipo;
    }

    public String getAmbito() {
        return ambito;
    }

    public int getLinea() {
        return linea;
    }
}
```




Autora: Beatriz Iglesias

## 17 FUNCIONAMIENTO DE TablaSimbolos

TablaSimbolos utiliza una lista de objetos Simbolo.

Sus métodos principales son:

agregarSimbolo()
Agrega un nuevo símbolo.

existeAgente()
Comprueba si un agente ya fue registrado.

buscarAgente()
Busca un agente por nombre.

existeEnAmbito()
Comprueba si un identificador existe dentro de un agente.

buscarEnAmbito()
Obtiene un símbolo dentro de un ámbito determinado.

limpiar()
Elimina los símbolos de la compilación anterior.

## 18. ANÁLISIS SEMÁNTICO

LEl análisis semántico se realiza después de construir el AST.

La clase encargada es:

Backend.semantico.AnalizadorSemantico

Su función es verificar que el programa, además de ser sintácticamente válido, cumpla las reglas de significado definidas por PromptZal.

Las principales validaciones son:

- Una variable debe ser declarada antes de utilizarse dentro del mismo agente.
- No pueden existir agentes con nombres duplicados.
- EJECUTAR únicamente puede hacer referencia a agentes existentes.
- EXPORTAR únicamente puede utilizar variables existentes en agentes ejecutados.

También se consideran las excepciones definidas por el lenguaje, como los identificadores utilizados como tipo de contenido en GENERAR y las unidades asociadas a valores numéricos.

## 19. MANEJO DE ERRORES

PromptZal maneja tres tipos de errores:

- Errores léxicos
- Errores sintácticos
- Errores semánticos

Cada tipo de error es registrado y posteriormente mostrado en la interfaz y en el reporte de errores.

| Tipo de error | Clase | Información almacenada |
|---|---|---|
| Léxico | ErrorLexico | Lexema, tipo, fila y columna |
| Sintáctico | ErrorSintactico | Encontrado, esperado, fila y columna |
| Semántico | ErrorSemantico | Descripción y línea |

### 19.1 Errores léxicos

Son detectados por JFlex cuando se encuentra una secuencia que no pertenece al lenguaje.

Ejemplos:

CARACTER_NO_RECONOCIDO
CADENA_SIN_CERRAR
DIRECTIVA_NO_RECONOCIDA
COMENTARIO_BLOQUE_SIN_CERRAR

El error se registra y el análisis continúa siempre que sea posible.

19.2 Errores sintácticos

El parser registra un error cuando el token encontrado no corresponde con el esperado por la gramática.

Cada error almacena:

• token encontrado
• token esperado
• fila
• columna

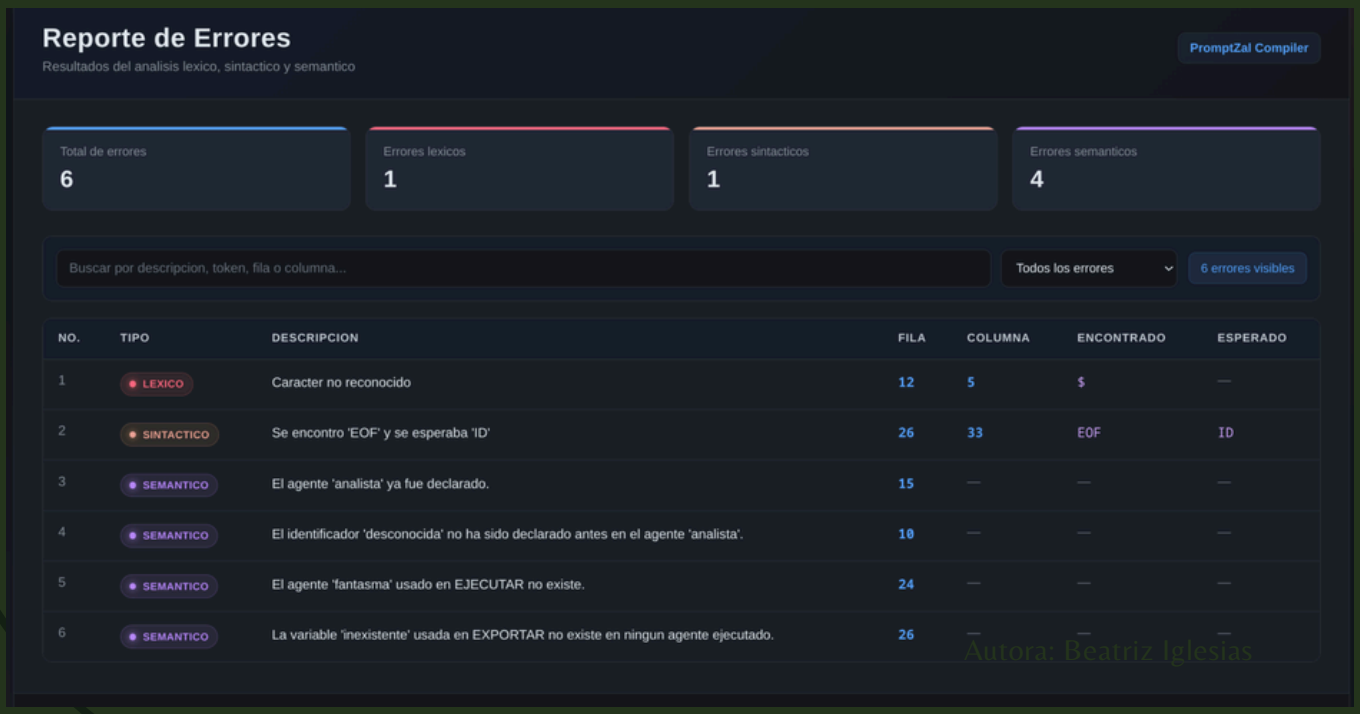
Después del registro, el parser utiliza recuperación mediante modo pánico para continuar el análisis.

19.3 Errores semánticos

Se generan cuando la estructura sintáctica es válida pero no cumple las reglas semánticas del lenguaje.

Entre los casos controlados se encuentran:

• variables no declaradas
• agentes duplicados
• EJECUTAR con un agente inexistente
• EXPORTAR con una variable inválida




Autora: Beatriz Iglesias

## 20. GENERACIÓN DEL PROMPTZAL

La generación del resultado final se encuentra en:

Backend.generador.GeneradorPromptzal

El generador recibe directamente:

Programa programa

Por lo tanto, el prompt se construye recorriendo el AST y no leyendo nuevamente el código fuente.

Flujo:

```
Programa / AST
    |
    v
GeneradorPromptzal
    |
    v
Prompt final
```

El método principal es:

generar(Programa programa)

Este método agrega:

1. Directivas.
2. Agentes ejecutados.
3. Exportaciones.

```java
public String generar(Programa programa) {
    if (programa == null) {
        return "";
    }

    StringBuilder prompt = new StringBuilder();

    prompt.append("# Prompt generado con PromptZal\n\n");

    agregarDirectivas(programa, prompt);
    agregarAgentesEjecutados(programa, prompt);
    agregarExportaciones(programa, prompt);

    return prompt.toString();
}

private void agregarDirectivas(
        Programa programa,
        StringBuilder prompt) {

    String modelo = buscarDirectiva(programa, "@modelo");
    String rol = buscarDirectiva(programa, "@rol");
    String formato = buscarDirectiva(programa, "@formato");
```

**PROMPT GENERADO**

```
# Prompt generado con PromptZal

Modelo destino: claude-sonnet-4-6
Rol general: analista
Formato de respuesta: markdown

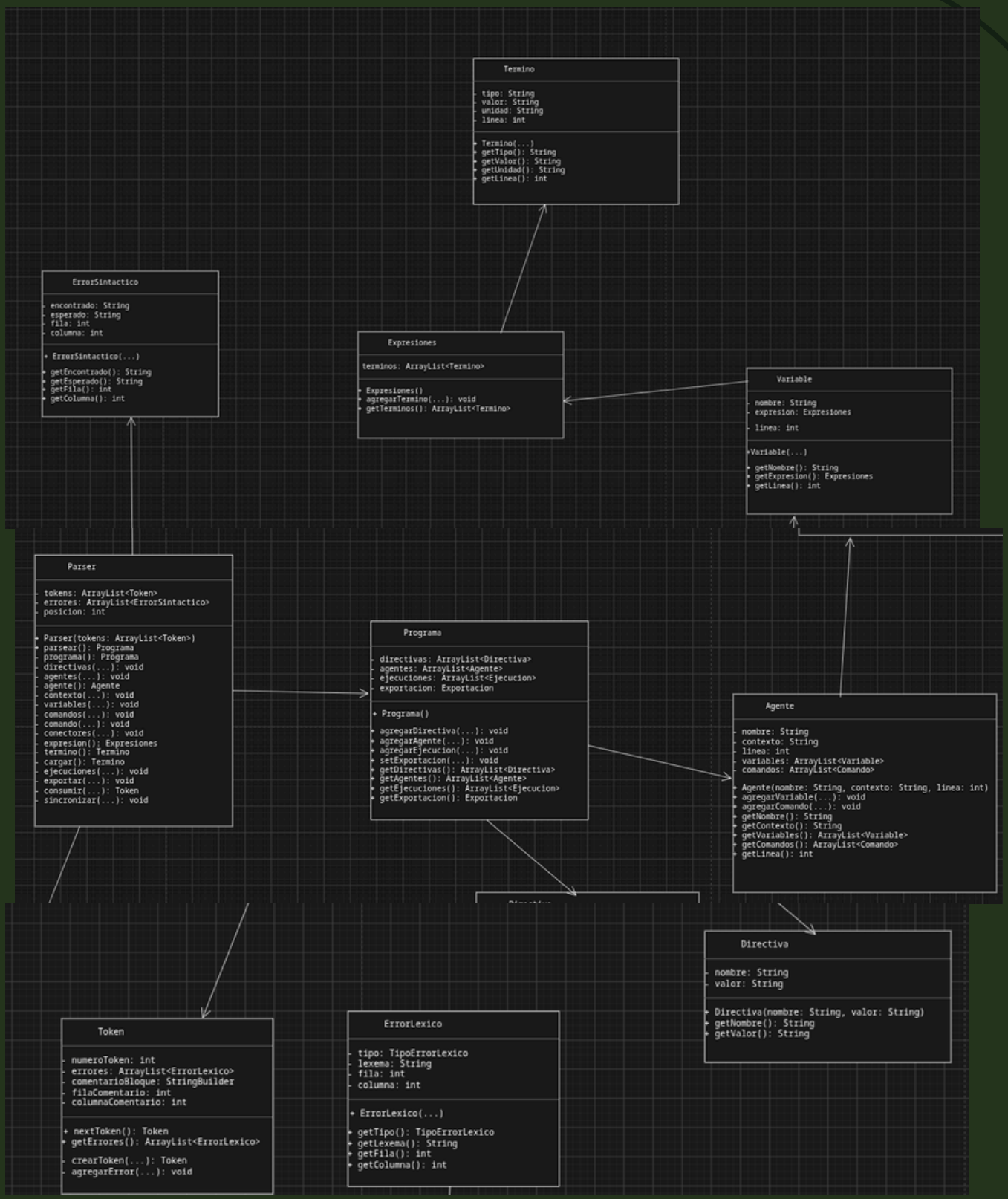
## Agente: analista

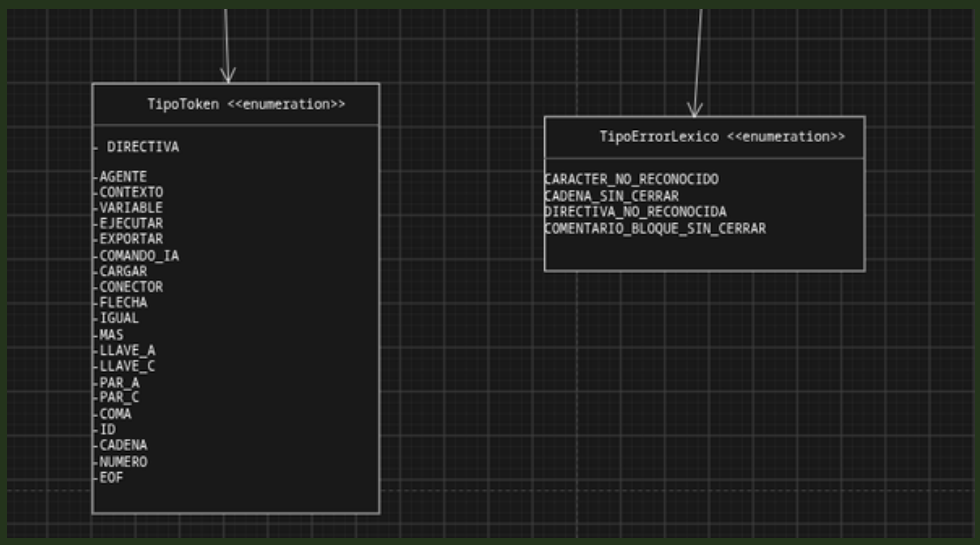
Contexto: Eres un analista encargado de
estudiar datos

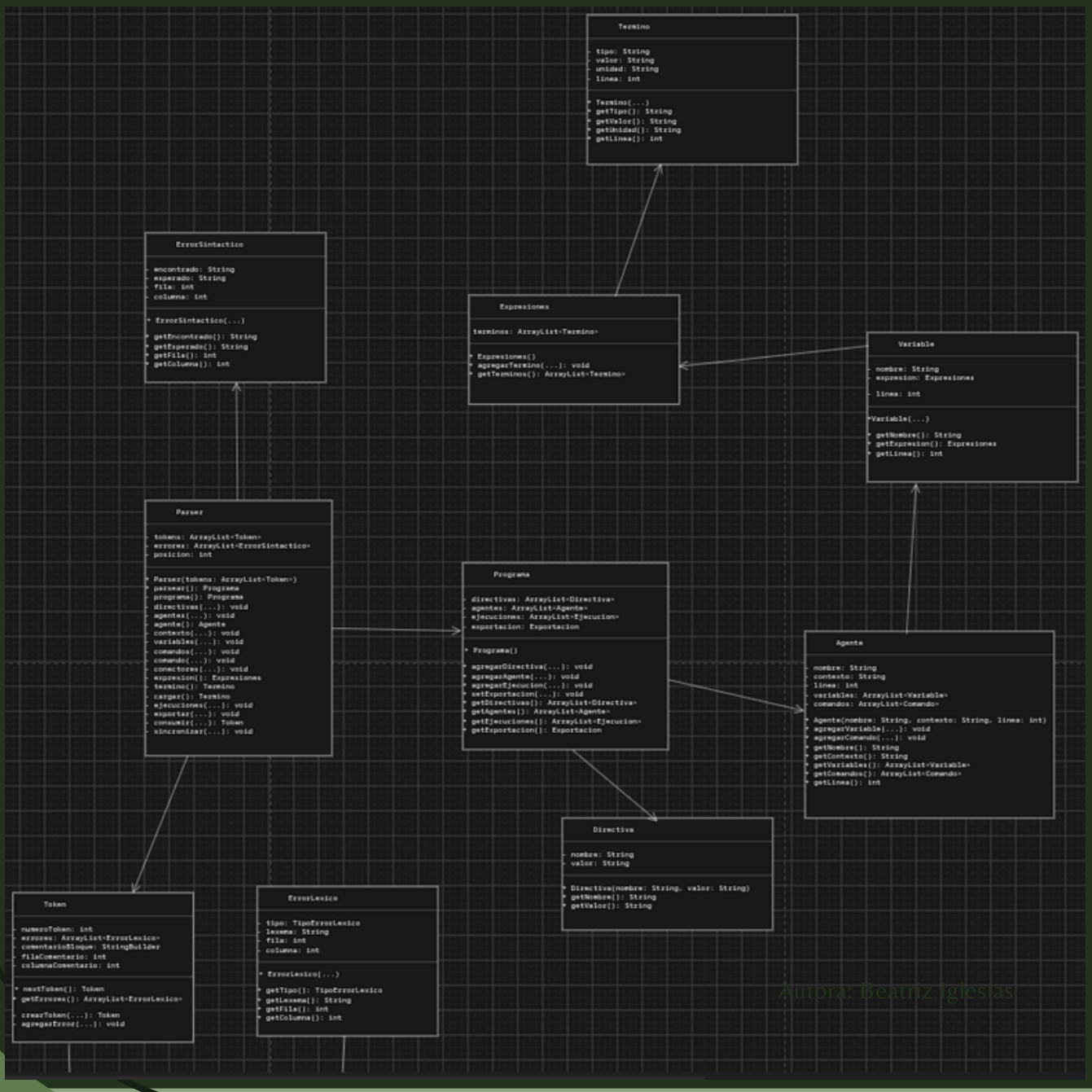
Variables disponibles:
- datos: contenido del archivo "datos.csv"
(adjuntar el archivo)

Instrucciones:
1. Basandote en datos, responde: Cuales son
los principales hallazgos? Guarda la respuesta
como respuesta.
2. Resume respuesta en un maximo de 80
```

DIAGRAMA DE CLASES






Autora: Beatriz Iglesias